

**ListaContigua:** Desarrollar la clase “ListaContigua” que representa una lista de números enteros y que además permite realizar operaciones sobre ellos.

La clase **ListaContigua** tiene los siguientes atributos:

- **int n:** atributo que almacena de forma privada el número de elementos almacenados actualmente en la lista contigua.
- **int capacidad:** atributo que indica el máximo número de elementos que se pueden almacenar en la lista.
- **int incremento:** atributo que indica el número de posiciones que se incrementa/decrementa la capacidad de la ListaContigua cuando es necesario.
- **int \*vector.** Puntero a un array de enteros que permitirá almacenar **capacidad** elementos de tipo int. Este array deberá ser reservada de forma dinámica en sus constructores y ser liberada en su destructor.

Para esta clase deberán implementarse los siguientes métodos públicos:

- **Constructor por parámetros. ListaContigua(int incremento)** Inicializará los atributos n, capacidad e incremento así como el puntero al vector de enteros.
- **Destructor.** Se encargará de liberar la memoria que fue reservada de forma dinámica para almacenar el vector.
- **int getValor(int pos).** Devuelve el elemento de la lista contigua que se encuentra en la posición **pos**.
- **void int setValor(int pos, int val).** Modifica el elemento de la lista contigua que se encuentra en la posición **pos** por el valor **val**. **OJO:** Este elemento tenía que haberse insertado anteriormente
- **int getN().** Devuelve el tamaño actual de la lista contigua.
- **int getCapacidad().** Devuelve la capacidad de la lista contigua (Máximo número de elementos que puede albergar).
- **void insertar(int pos, int val).** Inserta un nuevo elemento en la posición **pos** de la lista con valor **val**, dejando primero un hueco para introducirlo (desplazando los elementos a la derecha con **memmove**). En el caso de que al insertar se alcance la máxima capacidad del vector deberá incrementarse esta en la cantidad **incremento** usando **realloc**.
- **void eliminar(int pos).** Elimina el elemento que se encuentra en la posición **pos** y por tanto deberá desplazar a la izquierda todos los elementos que se encuentren a su derecha mediante **memmove**. Si al eliminar este elemento el número de elementos del vector es menor o igual que (**capacidad-2xincremento**), se deberá reducir la capacidad en incremento elementos (**capacidad-incremento**).
- **void concatenar(ListaContigua \*lista).** Concatena la lista indicada como parámetro al final de nuestra lista (almacenada en vector).
- **int buscar(int num).** Busca un elemento en la lista contigua con valor igual a **num** y retorna su posición o -1 si no se ha podido encontrar.

## Algoritmos y estructuras de datos

---

Utilizar el fichero main.cpp proporcionado a través de Blackboard para realizar las pruebas necesarias y para enviar al corrector automático. Este programa permite realizar las siguientes operaciones:

- N: permite crear una nueva lista indicando el incremento.
- I: permite insertar un valor en la lista en una posición.
- E: permite eliminar un elemento de la lista.
- V: permite ver el valor de un elemento de la lista.
- T: permite ver todos los valores de la lista.
- S: permite modificar un valor de la lista.
- L: informa sobre la longitud actual de la lista.
- M: informa sobre la capacidad máxima de la lista.
- C: permite concatenar nuestra lista con otra con n elementos.
- B: permite buscar un valor en la lista.
- F: termina.

NOTA: Para cada uno de los métodos implementados se deberá incluir una pequeña descripción de su funcionamiento, sus precondiciones mediante `assertdomjudge` si las hubiera y el análisis de su complejidad temporal y espacial. Esta información deberá incluirse en la cabecera de cada función.